

컴퓨터비전(AI 응용)

3D 렌더링



3D 렌더링 개론

가상 세계의 구성 요소와 목표



기본 단위: 2D vs 3D

삼각형과 메쉬(Mesh)의 이해



NeRF의 기초

문제 정의와 핵심 작동 원리



NeRF 구현 심화

카메라 모델, Ray Casting, Tiny NeRF



Volume 렌더링

연속적 공간 표현과 장단점 분석



Gaussian Splatting

개요, 원리 및 NeRF와의 비교



확률 모델과 ML

다변량 가우시안 분포의 활용



요약 및 마무리

전체 기술 흐름 정리

가상 세계를 인간의 시각 체계처럼 만들기

가상 세계의 구성



GOAL

'인간의 시각 체계를 모사하는' 가상 세계를 만드는 것



STEP 1

가상 카메라 배치

시점과 보는 방향 설정



STEP 2

픽셀 그리드 삽입

화면 평면 구성



STEP 3

오브젝트 배치

3D 물체 생성



STEP 4

광원 추가

빛과 그림자 연산



FINISH

최종 렌더링

이미지 생성 완료

"광원이 빛을 방출하고, 물체가 반사하고, 카메라가 이를 받아들이는 과정을 계산"

3D 렌더링은 어디에 쓰일까?



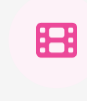
게임 그래픽

실시간 렌더링, 물리 엔진



메타버스

가상 사회, 아바타 상호작용



영화 / 광고 CG

VFX, 고품질 오프라인 렌더링



3D Rendering

AR / VR

증강현실, 몰입형 경험



건축 / 설계

디지털 트윈, 시각화



자율주행 시뮬레이션

가상 환경 주행 학습



STEP 1

현실 세계



STEP 2

카메라 촬영



STEP 3

3D 복원 (NeRF)



STEP 4

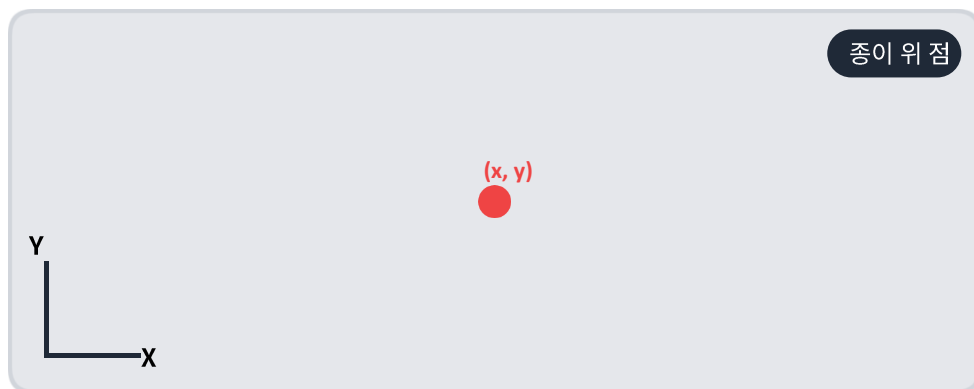
가상 세계 재현

2D와 3D의 차이

평면에서 입체 공간으로의 확장

DIMENSION 02

2D

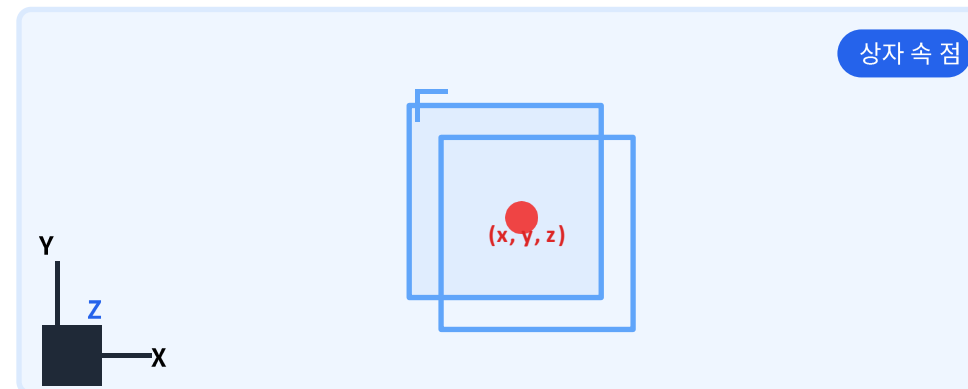


- ✓ 좌표계: X, Y 축만 존재
- ✓ 특징: 평면 (Flat Surface)
- ✗ 깊이(Depth) 없음
- ✗ 원근감 표현 한계

VS

DIMENSION 03

3D



- ✓ 좌표계: X, Y, Z(깊이) 축 존재
- ✓ 특징: 입체 공간 (Volume)
- ✓ 거리에 따른 원근감 발생
- ✓ 카메라 위치에 따라 모습 변화

3D 공간 내의 객체(Object) 다루기

컴퓨터는 연속적인 곡면을 직접 이해할 수 없습니다.
따라서 모든 객체는 **가장 단순한 단위**로 쪼개져야 합니다.

📍 점들의 집합 (Vertices)

객체(Object)는 3차원 공간상의 좌표(x, y, z)를 가진 점들의 집합으로 표현됩니다. 이 점들이 모여 형태를 이룹니다.

📐 기본 단위: 삼각형 (Triangle)

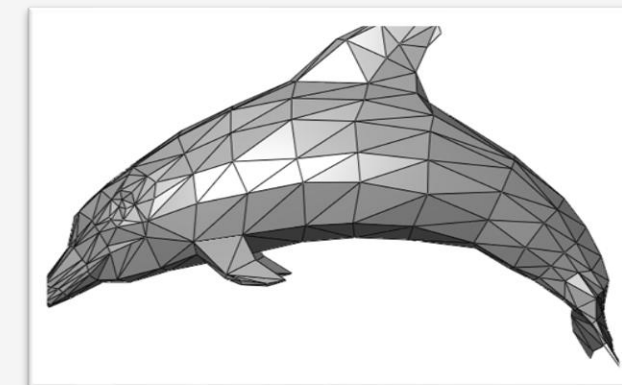
점들을 연결해 면(Face)을 만들 때, 그 최소 단위는 삼각형입니다. **현대 3D 그래픽스는 거의 모든 객체를 삼각형으로 처리합니다.**

📐 왜 하필 삼각형인가?

3개의 점은 **항상 하나의 평면 위에 존재**합니다. 4개 이상의 점은 뒤틀릴 수 있지만, 삼각형은 왜곡 없이 완벽한 평면을 보장합니다.

⚙️ GPU 최적화

삼각형은 수학적으로 가장 단순하여 연산 속도가 매우 빠릅니다. 그래픽 카드(GPU)는 삼각형 처리에 고도로 최적화되어 있습니다.



왜 삼각형이 기본일까?



완벽한 평면성

3점은 항상 하나의 평면
절대 뒤틀리지 않음



계산 단순화

법선 벡터(Normal) 및
조명 계산이 가장 쉬움

GPU 최적화

모든 그래픽 카드는
삼각형 처리에 특화됨



표면 근사

가장 단순한 도형으로
모든 곡면 표현 가능



Triangle
기본 단위

사각형 (Quad)의 문제



점 하나만 움직여도 평면이 깨져버림
(Normal 정의 불가)

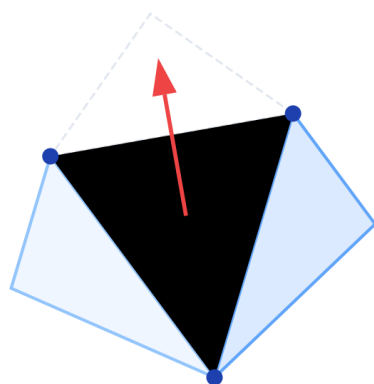
삼각형 (Triangle)의 장점



어떤 점을 움직여도 항상 평면 유지
(안정적 계산)

현실의 매끄러운 물체를 디지털 세상으로 옮기기 위해, 우리는 **Polygon Mesh**라는 껍데기를 사용합니다.

Mesh 구조 시각화



- Vertex (정점)
- Face (면)
- Edge (모서리)
- Normal (법선)

메시의 3요소 (V-E-F)

메시는 **점(Vertex)**, 이를 잇는 **선(Edges)**, 그리고 선들이 닫혀 만들어진 **면(Faces)**으로 구성된 그물망 구조입니다.

연속 곡면의 근사 (Approximation)

컴퓨터 메모리는 무한하지 않으므로, 부드러운 곡면을 유한한 개수의 작은 평면(삼각형)들로 쪼개어 근사적으로 표현합니다.

표면 속성 (Attributes)

단순 형상 외에도 빛 반사 방향을 결정하는 **법선(Normal)** 벡터와, 2D 이미지를 입히기 위한 **UV 좌표** 정보를 포함합니다.

주요 파일 포맷

.OBJ (구조 중심), .FBX (애니메이션 포함), .GLTF (웹/모바일 최적화)

NeRF(Neural Radiance Fields)는 2D 이미지를 이용해 **3D 공간을 복원**하고 새로운 시점의 이미지를 생성하는 기술입니다.

NeRF Pipeline

Inputs → 3D Representation → New View

📷 입력 (Input)

보정된 **다중 시점 이미지**(Calibrated Images)와 각 이미지의 **카메라 포즈**(위치, 방향) 정보를 입력받습니다.

🖼️ 출력 (Output)

학습에 사용되지 않은 **새로운 시점(Novel View)**에서의 이미지를 합성해냅니다.
마치 그 자리에 가서 찍은 것처럼 자연스러운 장면을 생성합니다.

🧠 핵심 원리 (Core Concept)

공간상의 모든 점에 대해 **밀도(Density)**와 **색상(Radiance)**을 정의하는 연속적인 함수를 딥러닝 네트워크로 학습합니다.

NeRF 문제 정의

NeRF(Neural Radiance Fields)는 2D 이미지를 이용해 **3D 공간을 복원**하고 새로운 시점의 이미지를 생성하는 기술입니다.

NeRF Pipeline

Inputs → 3D Representation → New View

Input

Calibrated 된 이미지 세트



Calibrated image set: 카메라 보정(Calibration) 과정을 거친 이미지
- 렌즈 왜곡, 시야 각, 위치, 방향을 모두 알고 있다는 것을 의미함

Output

Novel view 로 그려진 3D 장면 표현



novel view: 기존의 카메라로 촬영하지 않은 새로운 시점을 의미함.
즉, Input 사진에서 존재하지 않는 새로운 각도의 형태를 추론한 것

NeRF를 한 문장으로 설명한다면?

"여러 장의 사진으로 3D 공간을 추측해, 찍지 않은 아무 각도에서나 이미지를 만들어내는 AI"

NeRF: Neural Radiance Fields



입력: 사진 더미

위치를 아는 여러 장의 2D 이미지



비유: 사람의 상상

"친구 얼굴을 이리저리 보면 옆모습도 상상되죠?"



AI
Network

출력: 새로운 시점

Novel View Synthesis (임의 시점 렌더링)



원리: 공간 추측

빈 공간의 밀도와 색을 AI가 채워넣음



다중 사진 (2D)



신경망 학습 (AI)



3D 볼륨 표현



새로운 시점 (Novel View)

NeRF의 작동 원리 - 3가지 핵심

NeRF는 복잡한 3D 공간을 **인공지능 네트워크**로 표현하고, 이를 통해 새로운 시점의 이미지를 합성합니다.



Neural Volumetric 3D Scene Representation

3D 장면을 직접 저장(메시/복셀)하는 대신, **MLP 신경망(함수)**으로 표현합니다. 공간상의 위치(x,y,z)와 보는 방향(θ, ϕ)을 입력하면 **색상**과 **밀도**를 출력합니다.



Differentiable Volumetric Rendering

신경망이 예측한 공간 정보를 모아 **2D 이미지로 변환(렌더링)**합니다. 이 과정이 수학적으로 미분 가능하여, 딥러닝 학습이 가능해집니다.



Optimization via Analysis-by-Synthesis

"합성을 통한 분석" 방식입니다. 렌더링된 이미지가 **실제 사진과 같아지도록** 오차(Loss)를 계산해 신경망을 스스로 업데이트합니다.



3D Representation

$F(x, d) \rightarrow (RGB, \sigma)$



Volume Rendering

광선(Ray) 적분

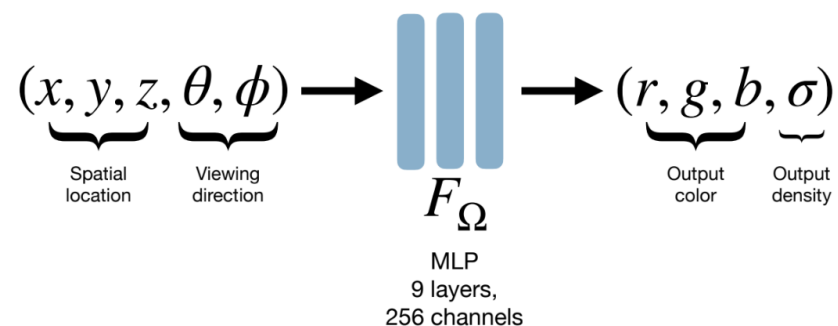
Update Weights



Optimization

Loss 최소화 (Backprop)

Neural Volumetric 3D Scene Representation



- (x, y, z) : 모델의 입력. 3차원 공간의 한 점이 통과하는 위치
- (θ, ϕ) : 관찰자의 시점과 방향(카메라가 보는 각도)
- F_{Ω} 는 학습 가능한 파라미터를 가진 함수
- 여기서는 9개의 레이어, 256개 채널을 가진 MLP를 의미함
- (r, g, b, a) 는 각각 RGB 채널 값과 투명도를 의미함

Different Volumetric Rendering Function



- Volumetric : 연속되는 voxels
- Voxels : 공간을 일정 크기로 나눈 3차원 큐브
- 일반적인 3D 그래픽에서의 Mesh(점, 선, 면)가 아닌 공간의 모든 점에 대한 색과 밀도 표현

Optimization Via Analysis-by-synthesis



- 결과를 합성(렌더링) 하며 물체를 추정하는 과정
- **Input**으로 여러 시점의 실제 이미지를 가지고 있을 때, **신경망을 통해 알아낸 색상과 밀도**를 바탕으로 광선을 따라 이미지를 합성
- 합성된 이미지와 실제 이미지의 오차를 줄이도록 최적화

Neural Volumetric Scene Representation

NeRF는 3D 장면을 **연속적인 함수**로 정의합니다.
인공신경망이 이 함수 $F(x, d) \rightarrow (c, \sigma)$ 를 학습합니다.

➡ 입력 (Input): 5D 좌표

공간상의 3D 위치 $\mathbf{x}$ (x, y, z)와 바라보는 2D 시점 방향 $\mathbf{d}$ (θ, ϕ)를 입력으로 받습니다.

➡ 출력 (Output): 색상과 밀도

해당 위치에서의 방사 색상 $\mathbf{c}$ (R, G, B)와 투명도를 결정하는 부피 밀도 σ (Sigma)를 출력합니다.

🧠 MLP + Positional Encoding

완전 연결 신경망(MLP)을 사용하며, 고주파수 디테일(선명함)을 표현하기 위해 좌표에 **위치 인코딩**을 적용합니다.

👁️ 시점 의존 효과 (View-Dependent)

보는 방향(d)을 입력으로 받기 때문에, 각도에 따라 달라지는 반사광(Specular)이나 재질감을 표현할 수 있습니다.

Scene Representation Network

좌표를 입력받아 색과 밀도를 출력

Differentiable Volumetric Rendering

미분 가능한 볼륨 렌더링: 공간을 적분하여 색상을 얻다

Volume Rendering Equation (Discretized)

$$C(r) = \sum T_i \cdot (1 - \exp(-\sigma_i \delta_i)) \cdot c_i$$

이산화된 합으로 표현하여 컴퓨터 연산 가능

$C(r)$

최종 픽셀 색상

광선(Ray) r 을 따라 렌더링된 결과값

T_i

누적 투과도 (Transmittance)

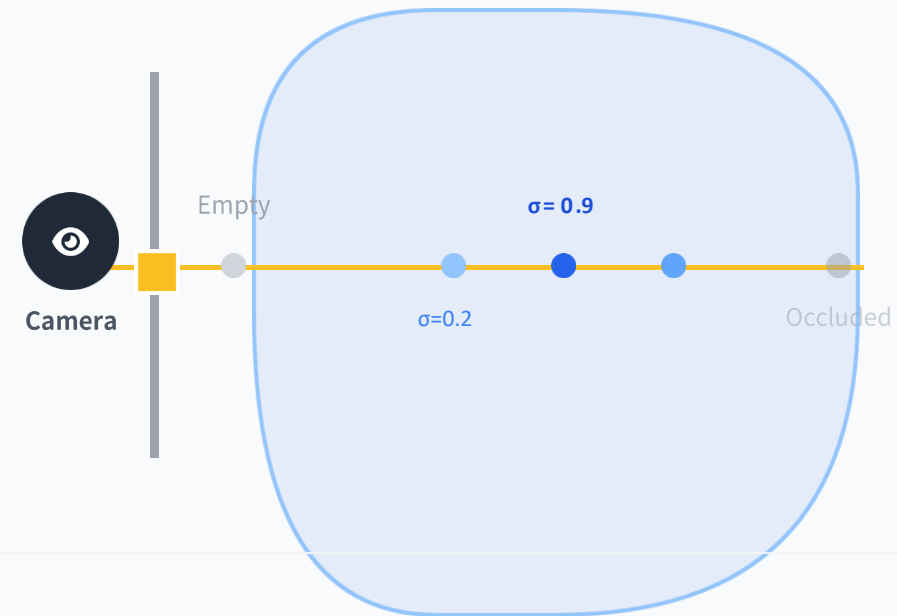
이전 구간까지 빛이 차단되지 않고 도달할 확률

σ_i

부피 밀도 (Volume Density)

해당 공간(점)에 입자가 존재할 확률 (불투명도)

Ray Marching Visualization



i 미분 가능성 (Differentiability)

이 과정의 모든 연산은 미분 가능합니다.
따라서 렌더링된 이미지와 실제 이미지의 차이(Loss)를 역전파(Backpropagation)하여 신경망을 학습시킬 수 있습니다.

Optimization via Analysis-by-Synthesis

NeRF는 정답 이미지와 AI가 만든 이미지의 차이를 줄이는 방식으로 스스로 학습합니다. 이를 '합성에 의한 분석'이라고 합니다.

✂ 손실(Loss) 최소화

실제 촬영된 정답 이미지(Ground Truth)와 렌더링된 합성 이미지 간의 색상 차이(MSE Loss 등)를 계산하여 오차를 최소화합니다.

↺ 역전파 (Backpropagation)

렌더링 과정 전체가 미분 가능하므로, 계산된 손실(Loss)을 역전파하여 MLP 신경망의 파라미터를 업데이트합니다.

Optimization Loop
정답과의 오차를 줄이며 반복 학습

⌂ 주의: 느린 수렴 속도

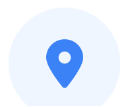
모든 픽셀에 대해 광선을 쏘고 적분해야 하므로 학습에 많은 반복(Iteration)이 필요하며, 수렴 속도가 느리다는 단점이 있습니다.



핵심 정의

“ 세상의 **모든 시점**에서 보이는 **빛의 정보(색상, 세기)**를 하나의 거대한 함수(지도)로 정의한 것 ”

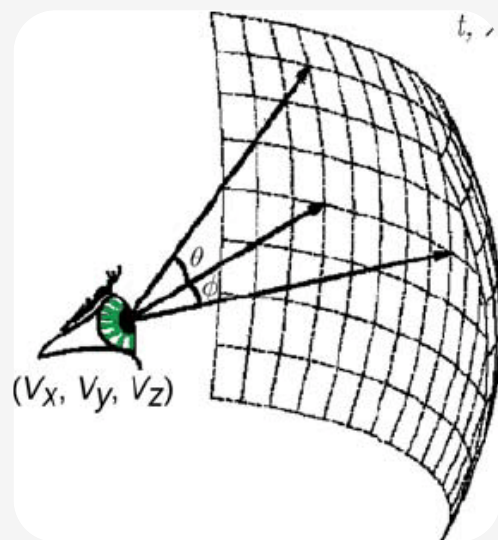
NeRF는 이 7차원 함수를 5차원(x, y, z, θ, ϕ)으로 단순화하여 학습합니다.



어디서 보는가?

 (x, y, z) : 3차원 위치

어떤 방향인가?

 (θ, ϕ) : 시선 각도

어떤 색인가?

 λ : 빛의 파장

언제 보는가?

 t : 시간

- 우리가 눈(카메라)으로 어떤 장면을 볼 때,

- 어떤 위치에서

- 어떤 방향으로

- 어떤 파장의 빛이 어느 정도 세기로 오는지를 알 수 있다면

어떤 각도에서든(input에 없는 각도라도) 그 물체를 본 모습을 상상할 수 있다.

• 플레노픽 함수(Plenoptic Function)

모든 가능한 시점, 방향, 시간, 파장에 따른 빛의 세기 변화를 함수로 표현함

- 7차원의 function 축약되어 표현됨 $I_{PF} = P(\theta, \phi, \omega, \tau, V_x, V_y, V_z)$

- $P(V_x, V_y, V_z)$: 3차원 공간 내에서의 위치

- $P(\theta, \phi)$: 광선의 방향(방위각과 고도)

- $P(\omega, \tau)$: 광선의 파장과 주파수(ω), 시간(τ)

NeRF는 이 7차원 함수를 5차원(x, y, z, θ, ϕ)으로 단순화하여 학습합니다.

The Plenoptic function

모든 것을 보는 함수: 7차원에서 5차원로의 단순화

Original 7D Function

$$\text{Rad} = P(x, y, z, \theta, \phi, \lambda, t)$$

NeRF Simplified 5D Function

$$(c, \sigma) = F_{\theta}(x, y, z, \theta, \phi)$$

7차원 → 5차원 축소

Static Scene Assumption

x, y, z

3D 위치 (Position)

공간상의 좌표점

θ, ϕ

보는 방향 (Viewing Direction)

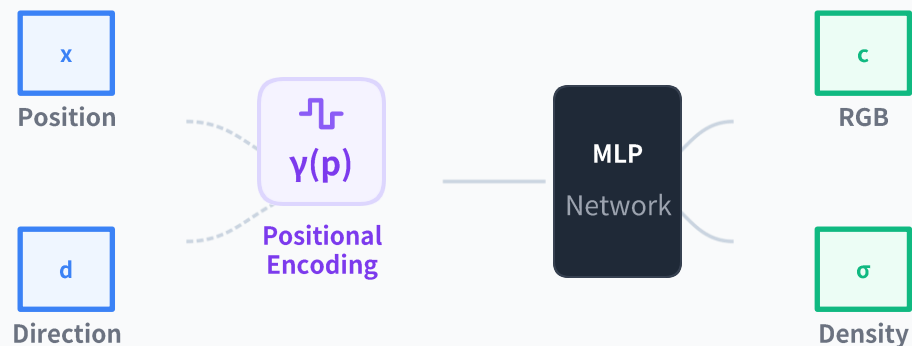
카메라가 바라보는 각도 (방위각, 고도각)

λ, t

파장 & 시간 (제거됨)

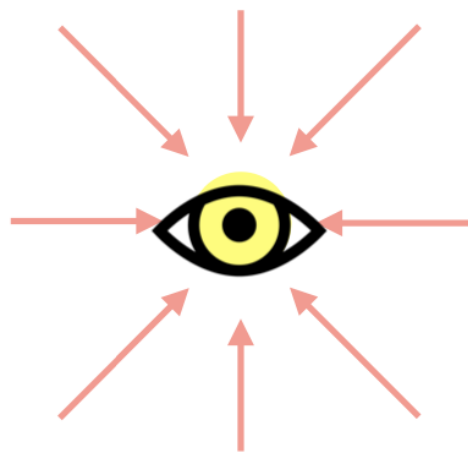
RGB 채널 통합 & 정적 장면 가정으로 생략

NeRF Input Processing Pipeline



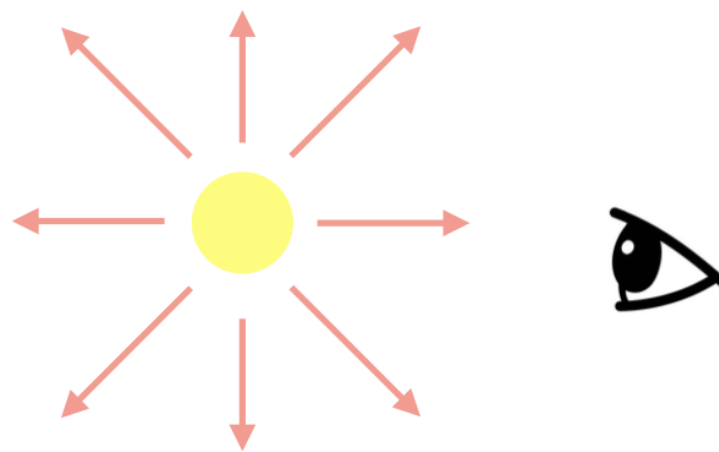
좌표 인코딩 (Positional Encoding)

단순 좌표(x, y, z)를 고차원 주파수 데이터로 변환하여, MLP가 **고주파 디테일(선명한 텍스처)**을 학습할 수 있게 돕습니다.



Plenoptic Function

- 관찰자(눈, 카메라)가 중심
- 외부에서 들어오는 모든 광선을 설명하려고 함

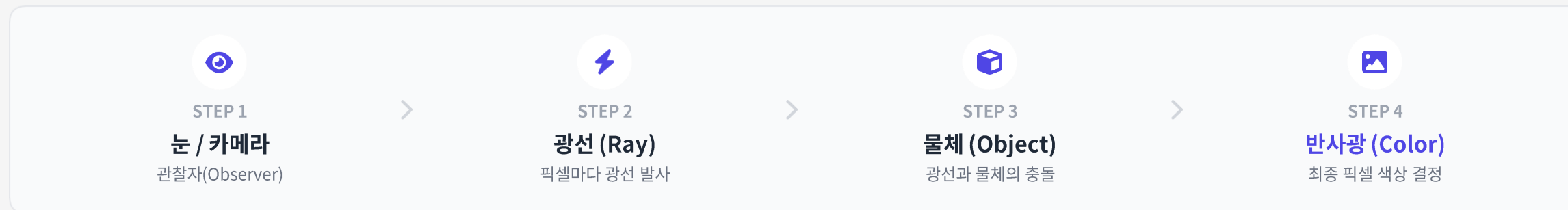
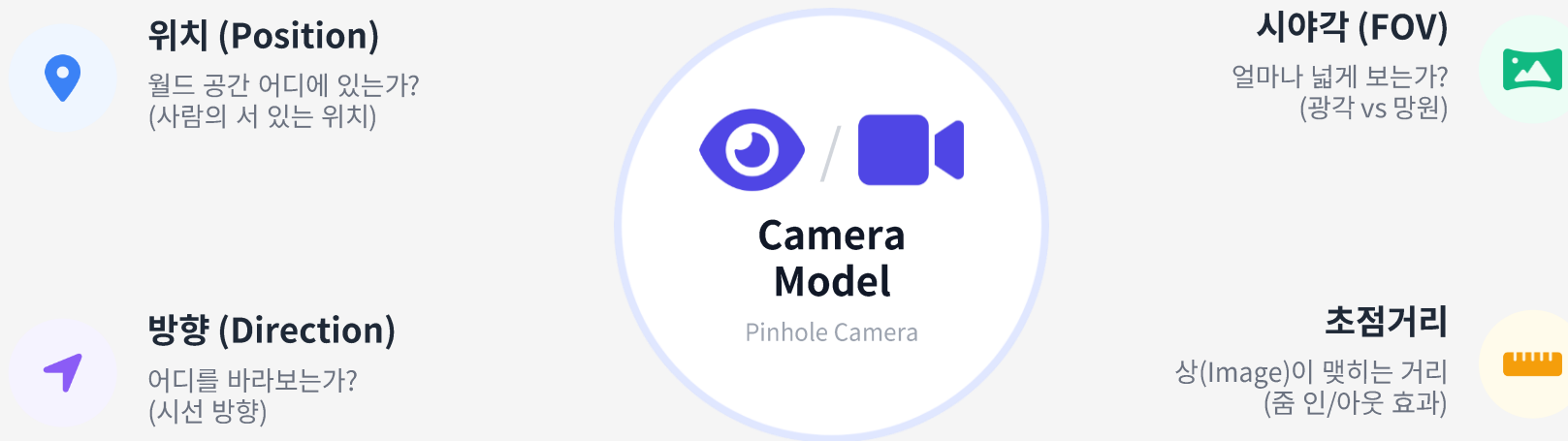


NeRF

- 장면, 혹은 물체가 중심
- 장면(물체)으로부터 나가는 빛의 세기를 학습하려 함

가상카메라는 사람 눈과 같다

우리가 세상을 보는 방식(눈)을 그대로 수학적으로 모델링한 것이 '가상 카메라'입니다.



컴퓨터 그래픽스는 인간 시각 시스템의 수학적 모방입니다.

Ray Origin과 Ray Direction 구현

3D 렌더링을 위해서는 2D 이미지의 각 픽셀이 3D 공간상의 어떤 방향으로 뻗어나가는 광선(Ray)인지 정의해야 합니다.

📷 카메라 파라미터 (Camera Parameters)

외부(Extrinsic): 월드 좌표계에서의 카메라 위치와 회전 (Pose)

내부(Intrinsic): 초점 거리(Focal Length), 주점(Principal Point) 등 렌즈 특성

Ray Casting

카메라에서 픽셀을 통해 쏘는 광선

→ 광선의 방정식 (Ray Equation)

$$r(t) = o + t \cdot d$$

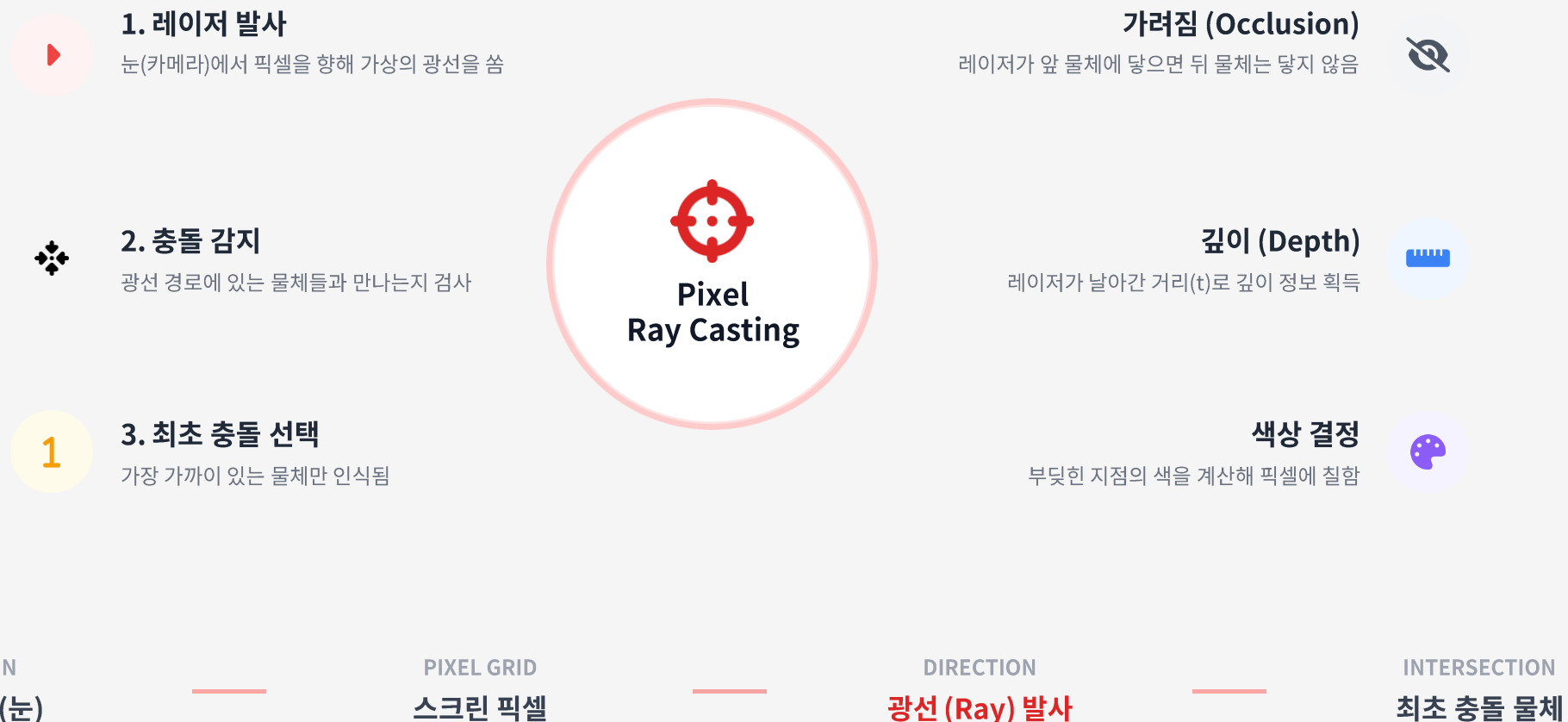
o : 원점(Origin), d : 방향(Direction), t : 거리

🌐 좌표계 변환 (Coordinate Transform)

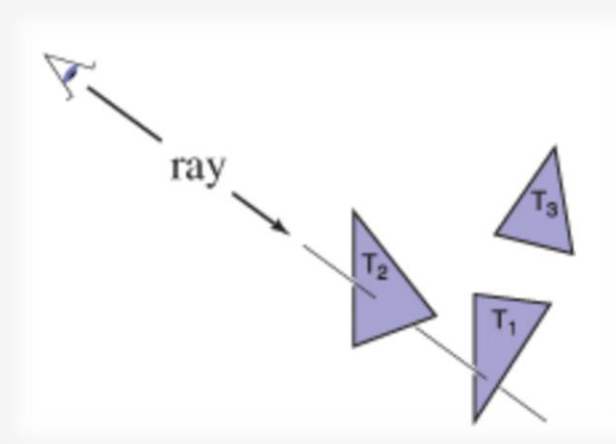
이미지 픽셀 좌표 (u, v)를 카메라 좌표계로 변환한 뒤, 다시 카메라의 자세 행렬을 곱해 **월드 좌표계의 방향 벡터**를 구합니다.

Ray Casting은 '레이저 포인터'와 같다

Ray Tracing과 NeRF의 기초가 되는 핵심 원리



- 렌더링(Rendering) : *convert colors, materials and so on.. into an image on camera plane.*
- Ray Tracing : 이미지에서 해당 픽셀의 위치에 보이는 객체가 무엇인지를 파악하는 것
- Ray Origin : 픽셀마다 광선을 쏘기 위한 시작점
- Ray Direction : 픽셀마다 광선을 쏘는 방향



그림의 경우, ray 에 닿는 것은 T2와 T1 두 개의 삼각형이지만
첫 번째로 닿는 객체(ray에서 보이는 객체)는 T2이므로
T1은 T2에게 가려졌다고 할 수 있다.

Tiny NeRF 구현

소형 MLP 네트워크로 NeRF의 전체 파이프라인을 가볍게 실습

Training Loop Pipeline

- 1 데이터 준비 (Data Loading)**
Images + Camera Poses 로드
- 2 광선 생성 (Ray Generation)**
각 픽셀에 대응하는 Ray Origin(o), Direction(d) 계산
- 3 MLP 추론 (Query Network)**
광선 위 샘플 포인트(xyz) → 색상(c) 및 밀도(σ)
- 4 렌더링 & 학습 (Volume Render & Loss)**
적분하여 이미지 생성 후 MSE Loss로 역전파

N_samples

64

광선 하나당 샘플링하는 점의 개수 (Coarse/Fine)

L_embed

10

Positional Encoding 차수 (고주파 디테일 복원용)

Tiny NeRF Architecture

Input (x, y, z)

Positional Encoding $\gamma(p)$
 $\sin(2^0\pi p), \cos(2^0\pi p), \dots$

MLP

Dense Layers + ReLU

RGB
Color

σ
Density

PyTorch 등으로 약 100줄 내외 코드로 구현 가능

NeRF는 물체를 단단한 표면이 아닌, **밀도와 색상을 가진 구름(Volume)**처럼 다룹니다.

공간의 정의 (Density & Color)

3D 공간의 모든 점(x, y, z)은 **밀도(σ)**와 **방사 복사도(c, 색상)** 값을 가집니다.
비어 있는 공간은 밀도가 0에 가깝고, 물체가 있는 곳은 높습니다.

광선 적분 (Ray Integration)

카메라에서 픽셀로 쏜 광선(Ray)을 따라 여러 점을 샘플링하고,
각 점의 불투명도와 색상을 누적(적분)하여 최종 픽셀 색을 결정합니다.

Volume Rendering
광선을 따라 밀도와 색을 적분

장점: 사실적 표현

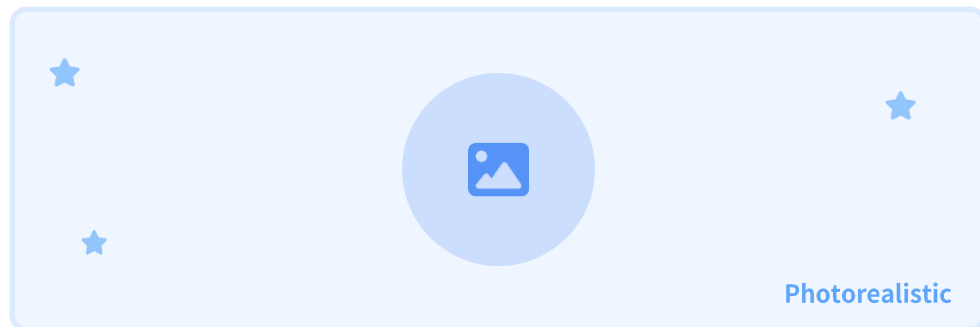
연기, 반투명한 물체, 복잡한 털 등 기존 메쉬(Mesh) 방식으로는 표현하기 어려운 디테일과
시점에 따른 빛 반사를 사실적으로 렌더링합니다.

단점: 높은 연산량

픽셀 하나를 만들기 위해 수백 번의 신경망 연산(샘플링)이 필요하므로,
렌더링 속도가 매우 느립니다. (실시간 처리가 어려움)

STRENGTHS

장점 (Pros)



매우 높은 품질

실사와 구별하기 힘든 고해상도 뷰 합성



연속적 볼륨 표현

미분 가능한 함수로 끊김 없는 공간 묘사



메쉬(Mesh) 불필요

복잡한 토폴로지 관리 없이 형상 표현

VS

LIMITATIONS

단점 (Cons)



느린 학습 및 추론

장면당 수 시간 학습, 렌더링에 수 초 소요



많은 메모리 소모

거대 MLP 네트워크와 샘플링 데이터 처리



실시간 렌더링 어려움

Ray Marching 연산 비용으로 인해 즉시 확인 불가

- Nerf는 신경망을 이용해 3D 장면을 연속된 볼륨 형태로 표현함
 - 장면은 픽셀이 아닌 공간의 밀도와 방사 복사도로 정의됨
 - 카메라에서 발사된 광선을 따라 일정한 간격으로 샘플링하며,
각 점의 색상과 밀도를 네트워크가 예측함
 - 이 점들의 값을 볼륨 렌더링 식을 통해 적분하여 최종 픽셀 색상을 계산함
 - 결과적으로 NeRF는 이미지 입력만으로도 3D 공간의 연속적 광학 정보를 복원함
-
- 장점 : 매우 사실적인 시점 변화가 가능함
 - 단점 : 계산량이 많고 실시간 렌더링이 어려움

NeRF의 느린 속도를 극복하기 위해 등장한 혁신적인 기술입니다.
장면을 **수많은 3D 가우시안 타원체**들의 집합으로 표현합니다.



3D 가우시안 (3D Gaussians)

공간상의 연속적인 볼륨 대신, 흐릿한 점(타원체)들을 겹쳐서 장면을 구성합니다.
점 구름(Point Cloud)의 확장된 형태라고 볼 수 있습니다.



핵심 파라미터

각 가우시안은 위치(Position), 공분산(Covariance, 크기/방향), 색상(Color), 불투명도(Opacity) 정보를 가집니다.

3D Gaussian Primitive
타원체로 표현된 장면의 기본 단위



래스터라이즈 기반 (Rasterization)

광선 추적(Ray Marching) 없이,
3D 타원체를 2D 화면에 투영하여 그리는(Splatting) 방식이라 렌더링 속도가 매우 빠릅니다.



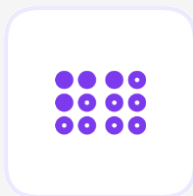
실시간 렌더링 가능

1080p 해상도에서 30fps 이상의 실시간 렌더링이 가능하여 VR/AR 등 상호작용 애플리케이션에 적합합니다.



PROCESS OVERVIEW

SfM 점구름에서 시작하여 파라미터를 최적화하고, 화면 공간에서 블렌딩하여 실시간 렌더링을 구현



STEP 1

초기화

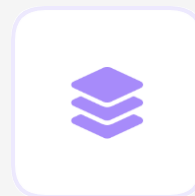
SfM / SLAM
점구름(Point Cloud)에서 시작



STEP 2

최적화

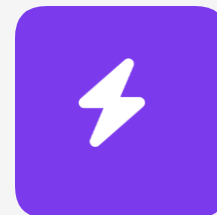
위치, 공분산, 색상, 불투명도
파라미터 업데이트



STEP 3

렌더링

화면 공간에서 가우시안을
정렬하고 중첩(Blending)



RESULT

GPU 친화적

매우 빠른 속도의
실시간 렌더링 구현 가능

- 장면을 **3D 가우시안 분포(점 구름 형태)** 로 표현함.
- 각 가우시안은 **위치, 색상, 크기, 방향** 등의 파라미터를 가짐.
- 가우시안들을 카메라 시점에서 **2D 이미지 평면에 투영**하여 렌더링함.
- **밀도 조절(adaptive density)** 과 **가시성 기반 렌더링(visibility-aware)** 으로 불필요한 계산을 최소화함.
- NeRF보다 계산 효율이 높고, **실시간 렌더링(real-time rendering)** 에 유리함.
- 다중 시점 이미지를 이용해 **고품질 3D 장면 복원** 가능.
- 단점: 가우시안 수가 많아지면 **메모리와 연산량이 증가**함.

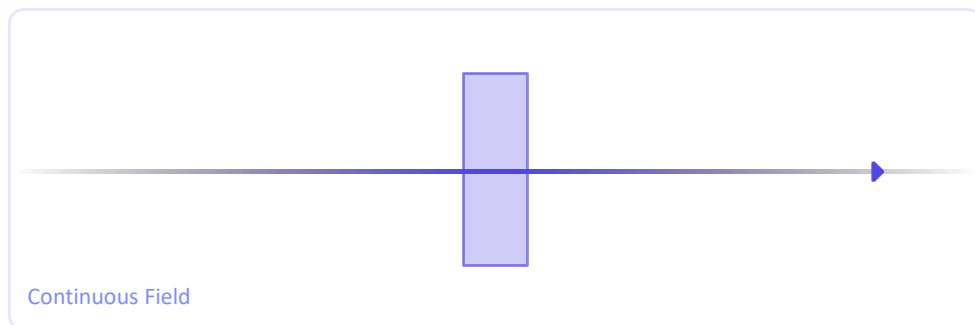
NeRF vs Gaussian Splatting

연속적인 볼륨 표현과 이산적인 점 기반 표현의 차이

VOLUMETRIC

NeRF

Neural Radiance Fields



Continuous Field

📦 **표현:** 연속 볼륨 (Continuous)

★ **화질:** 매우 높음 (사실적)

🕒 **학습 및 렌더링 속도가 느림**

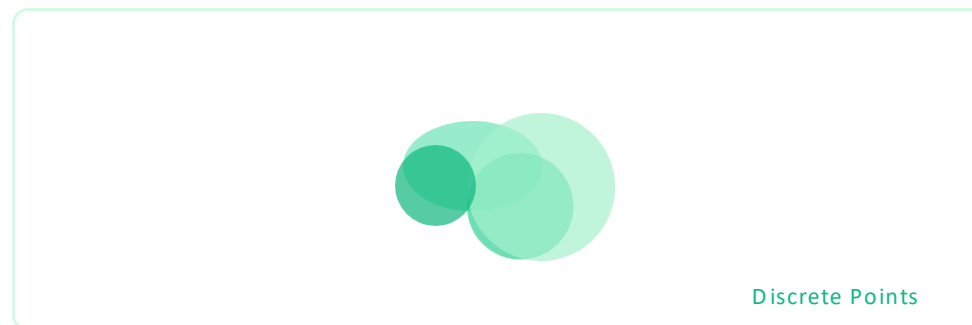
🚫 **실시간 처리 어려움**

VS

POINT-BASED

Gaussian

3D Gaussian Splatting



Discrete Points

📍 **표현:** 점 구름 (3D Gaussian)

⚡ **속도:** 매우 빠름 (실시간)

✅ **높은 품질 + 빠른 학습**

📦 **메모리 사용량 증가 가능성**

복잡한 데이터 분포를 설명하는 수학적 모델과 다양한 응용 사례

Probability Density Function (PDF)

$$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

▲ 데이터의 중심(위치)과 퍼짐(모양)을 동시에 정의

μ

평균 벡터 (Mean Vector)

분포의 중심 위치 (3D 공간에서 x, y, z 좌표)

Σ

공분산 행렬 (Covariance Matrix)

데이터의 회전, 크기(Scale), 방향 결정 (타원체의 모양)

🌐

등가 타원체 (Iso-surface)

같은 확률 밀도를 가진 점들을 이은 표면 (Gaussian Splatting의 알갱이)

Covariance(Σ)에 따른 모양 변화



Spherical



Diagonal



Rotated (Full Σ)

Gaussian Splatting은 이 'Rotated Ellipse'를 사용함

🔧 주요 활용 분야

VAE (생성 모델)

잠재 공간(Latent Space)의 데이터 분포를 가우시안으로 가정

Kalman Filter

로봇/자율주행의 상태 추정 및 위치 불확실성 모델링

GMM (클러스터링)

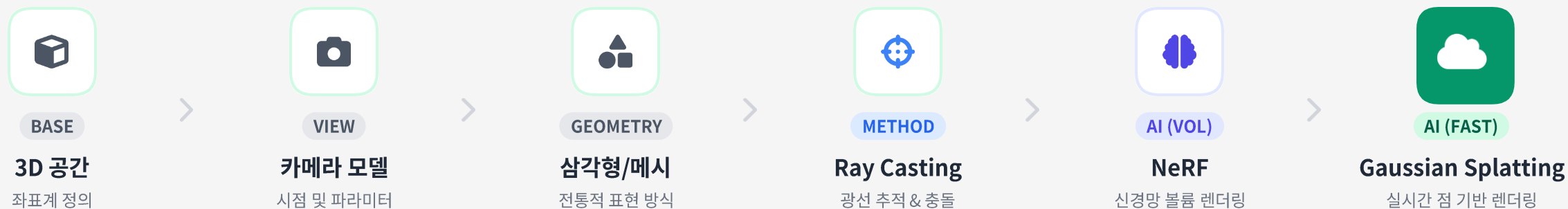
여러 개의 가우시안을 합쳐 복잡한 데이터 군집화

이상치 탐지

확률 밀도가 낮은 데이터를 비정상(Anomaly)으로 판별

- 다변량 가우시안 분포는 여러 변수 간의 확률 분포를 동시에 설명하는 확률 모델임
- 머신러닝에서는 데이터 분포 추정, 이상치 탐지, 베이지안 추론 등 다양한 확률 모델링에 활용됨
- VAE에서 잠재공간의 분포를 정규화하기 위해 다변량 가우시안을 사용함

"가상 공간 구축부터 최신 AI 렌더링 기술까지의 진화 과정"



강의 핵심 요약

3D 기본(삼각형)에서 시작해, 연속적인 볼륨을 학습하는 **NeRF**, 그리고 빠르고 정교한 **Gaussian Splatting**으로의 기술 흐름을 이해했습니다.



실습 및 학습 팁

복잡한 데이터보다는 작은 장면으로 **Tiny NeRF**부터 구현해보세요. 특히 입력 이미지의 **Camera Pose(위치/방향) 정확도**가 렌더링 품질을 결정짓는 가장 중요한 요소입니다.